

ENGINEERING CHANGE NOTE (ECN)

PROJECT: PROJECT-ASYNC updating to PROJECT-CE

ORIGINATOR: Prof. Kidd, Instructor

PURPOSE OF CHANGE: Cyclic Executive Implementation

DOCUMENT NUMBER: ECN 1.0

DATE OF PUBLICATION: 2/29/2026

LEAD ENGINEER: Kid Kidd, UMBC

TEAM LEAD: Nicholas McDonough

EXECUTIVE SUMMARY:

This engineering change notice documents the architectural changes made to project-ASYNC to create project-CE. The core software execution was altered to utilize a polled round-robin Cyclic Executive task manager. To create this change, a function pointer table was added, which is referenced during the execution loop to execute the correct function, adding greater ease when adding a new function to the execution loop. All customer and high-level requirements (C.1-C.6, HL.1-HL.6) have remained unchanged and are preserved through reference. Tests has changed marginally. Test T.1-T.7 remain the same same though test T.8 has been added to ensure that the functions within the function pointer table are called correctly.

HIGH-LEVEL DESCRIPTION:

Modified the main software control structure to use a cyclic executive round-robin, which utilized a task table, instead of a hardcoded loop.

REASON FOR CHANGE:

This change allows for a more reliable and flexible design, which will decrease labor and design costs for future extensions or modifications.

Changed Documents:

No previous documents were changed; all new documents replaced existing ones

Added Documents:

ECN1.0: Engineering Change Note for Cyclic Executive Implementation.

311ProjectReport - project 2

Removed Documents:

No documents were removed

Replaced Documents:

Proj1.ino was replaced with Proj2.ino: The document is relatively the same, with the added cyclic executive control within the main control structure.

Any Additional Design Considerations:

The task dispatcher relies on a defined function pointer type (typedef void (*taskFunction)()) and a global task array (functionTable[]) which stores the functions taskIO and taskUpdateLED. The loop function executes a standard for loop over NUMTASKS, calculated dynamically via sizeof(functionTable) / sizeof(functionTable[0]). The loop then appropriately executes the functions in order.